

PA#1 — Đề xuất và kế hoạch dự án BakeOrder

CSC13114 · Nhóm tối đa 3 thành viên · Nhóm 23120225–23120226–23120229

Thành viên: Trần Gia Cường — 23120225; Nguyễn Ngọc Đại — 23120226; Nguyễn Hà Đạt — 23120229.

Repository: <https://github.com/datkrb/BakeOrder>.

1. Vấn đề và người dùng

BakeOrder là ứng dụng web hỗ trợ cửa hàng bánh nhỏ chuyển hội thoại đặt bánh tiếng Việt thành phiếu đơn để kiểm tra. Người dùng đại diện là **chị Mai, chủ một tiệm bánh sinh nhật nhận đặt trước**, vừa tư vấn khách vừa tổng hợp đơn cuối ngày. Đây là persona giả định cần xác thực, không phải người đã được phỏng vấn. Hiện tại, chị đọc lại tin nhắn rồi chép yêu cầu vào sổ hoặc bằng tính.

Vấn đề: Khi khách sửa yêu cầu qua nhiều tin nhắn, chị Mai dễ chép nhầm phiên bản cuối của đơn, dẫn đến làm sai bánh hoặc giao sai giờ. Dự án tập trung vào bước tổng hợp và kiểm tra đơn trước khi sản xuất, không thay thế toàn bộ hoạt động của cửa hàng.

2. Tính năng LLM và chi phí khi sai

Một tính năng cốt lõi: từ một hội thoại của một đơn, LLM tạo phiếu nháp gồm loại bánh, kích thước, số lượng, chữ trên bánh, ngày/giờ nhận và hình thức nhận. Mỗi thông tin đi kèm câu tin nhắn làm căn cứ; yêu cầu thiếu hoặc mâu thuẫn được đánh dấu để người dùng kiểm tra. Không đủ căn cứ thì để trống và hỏi lại, không tự đoán.

Ví dụ: khách nhắn “bánh 16 cm, lấy 18h ngày 20/11”, sau đó “đổi 20 cm nhé”. Phiếu phải ghi 20 cm và giữ lịch nhận cũ; nếu khách chỉ nhắn “đổi sang thứ Bảy” mà thiếu ngữ cảnh ngày, hệ thống yêu cầu xác nhận ngày cụ thể. Chủ cửa hàng xem hội thoại cạnh phiếu, sửa và xác nhận rồi mới lưu thành đơn chính thức.

Ai chịu thiệt và bao nhiêu: lấy tình huống giả định một đơn bán 350.000 đồng, chi phí làm bánh 180.000 đồng. Nếu phát hiện sai kích thước sau khi làm, cửa hàng có thể mất thêm 180.000 đồng để làm lại; nếu hủy đơn, có thể phải hoàn 350.000 đồng đã thu và chịu chi phí bánh đã làm. Đây là hai kịch bản, không cộng gộp thành một khoản thiệt hại. Khách mất thời gian, và giao sau giờ tiệc có thể không khắc phục được bằng việc đổi bánh. Các con số sẽ được xác thực khi phỏng vấn. Trước sản xuất, lỗi thường có thể sửa trên phiếu; sau giao hàng, việc sửa dữ liệu không hoàn tác được thiệt hại.

Làm sao biết sai: dự kiến đánh giá bằng 60 hội thoại có đáp án được người đọc kiểm tra, gồm đổi yêu cầu, thiếu thông tin, mâu thuẫn và cách viết tắt. Dùng 40 hội thoại phát triển, giữ riêng 20 hội thoại đánh giá cuối. Theo dõi độ đúng từng trường, số đơn sai trường quan trọng và số trường người dùng phải sửa. Mục tiêu ban đầu: đúng ít nhất 95% trường quan trọng đã có thông tin và không tự điền các trường cố ý thiếu trong bộ đánh giá; đây là mục tiêu, chưa phải kết quả đạt được.

3. Phạm vi học kỳ

Làm: dán hội thoại văn bản tiếng Việt, một đơn mỗi lần; tạo phiếu nháp và chỉ ra căn cứ; cảnh báo thiếu/mâu thuẫn; sửa, xác nhận, lưu và xem danh sách đơn; đánh giá chất lượng trích xuất và ghi nhận thời gian xử lý, token, chi phí. Bản thử nghiệm chạy cho một cửa hàng với người vận hành tin cậy trên máy demo.

Không làm: kết nối Zalo/Facebook, ảnh hoặc giọng nói, tự trò chuyện với khách, tự xác nhận đơn, thanh toán, vận chuyển, quản lý kho, nhiều chi nhánh, triển khai công khai nhiều tài khoản. Yêu cầu dự ứng chỉ được giữ nguyên để người bán kiểm tra, không được AI kết luận về độ an toàn thực phẩm.

4. Kế hoạch qua sáu checkpoint

Lịch checkpoint: thứ Năm hằng tuần từ 15/10/2026 đến 19/11/2026. Trần Gia Cường phụ trách nhu cầu và đánh giá; Nguyễn Ngọc Đại phụ trách LLM/backend; Nguyễn Hà Đạt phụ trách giao diện và tích hợp. Mỗi mốc có một người chịu trách nhiệm cuối cùng, các thành viên còn lại hỗ trợ và rà soát.

Mốc	Công việc và kết quả dự kiến	Chủ trì	Hạn hoàn thành
CP1	Chốt vấn đề, phạm vi, đề xuất PA#1, repo và tự đánh giá.	Trần Gia Cường	15/10/2026
CP2	Phỏng vấn một chủ tiệm; chuẩn bị 20 hội thoại đầu; lập đặc tả khi đến mốc yêu cầu.	Trần Gia Cường	22/10/2026
CP3	Thử trích xuất có căn cứ, đánh dấu thiếu/mâu thuẫn; báo cáo lỗi và chi phí trên 40 mẫu phát triển.	Nguyễn Ngọc Đại	29/10/2026
CP4	Tích hợp luồng dán → nháp → sửa → xác nhận → lưu; demo và kiểm tra dữ liệu lưu.	Nguyễn Hà Đạt	05/11/2026
CP5	Hoàn thiện 60 mẫu; đánh giá 20 mẫu giữ riêng; thử với người dùng, sửa lỗi nghiêm trọng.	Trần Gia Cường	12/11/2026
CP6	Chốt bản demo, hướng dẫn chạy, kết quả đánh giá, giới hạn và thay đổi so với kế hoạch.	Nguyễn Hà Đạt	19/11/2026

Phụ thuộc chính: dữ liệu mẫu trước thử LLM; luồng xác nhận trước thử người dùng. Dự kiến hoàn thành công việc chính trước mỗi hạn chính thức ba ngày để sửa lỗi. Nếu độ chính xác chưa đạt ở CP3, giảm xuống đơn một loại bánh và ưu tiên kích thước, số lượng, lịch nhận; vẫn giữ xác nhận thủ công. Nếu yêu cầu chi tiết từng checkpoint thay đổi, nhóm cập nhật công việc trên repository và ghi lại lý do.

5. Hai rủi ro có thể khiến dự án thất bại

R1 — Trích xuất sai nhưng người dùng tin phiếu nháp: đặc biệt khi khách đổi ý hoặc dùng ngày tương đối; nếu lỗi thường xuyên, sản phẩm không tiết kiệm công kiểm tra. **Bắt đầu tuần này:** Nguyễn Ngọc Đại tạo 20 ca khó có đáp án, thử mô hình và phân loại lỗi; Nguyễn Hà Đạt phác thảo hiển thị căn cứ cạnh từng trường và bước xác nhận bắt buộc. Sau đó mở rộng bộ đánh giá, không dùng điểm “tự tin” của mô hình thay cho đối chiếu căn cứ.

R2 — Không tiếp cận được người dùng/dữ liệu thực tế: dữ liệu tự viết quá sạch có thể khiến kết quả thử không phản ánh nhu cầu. **Bắt đầu tuần này:** Trần Gia Cường liên hệ ba cửa hàng để xin một buổi trao đổi và ví dụ đã ẩn thông tin cá nhân, có sự đồng ý. Nếu sau bảy ngày chưa có người tham gia, nhờ một người từng nhận đơn bánh đánh giá 20 tình huống mô phỏng; ghi rõ giới hạn, không trình bày dữ liệu mô phỏng như dữ liệu thật. Không đưa tin nhắn nhận diện khách hàng vào repository.

6. Công nghệ và chi phí

- Next.js + TypeScript:** giao diện và API trong cùng dự án để tích hợp nhanh luồng phiếu nháp; chỉ gọi mô hình ở server để giữ kín API key.
- SQLite:** lưu đơn đã xác nhận và các lần chỉnh sửa trong bản demo một cửa hàng, không cần vận hành máy chủ cơ sở dữ liệu riêng.
- Google Gemini Developer API, gemini-2.5-flash-lite:** lựa chọn ban đầu cho trích xuất văn bản; đánh giá trên bộ mẫu trước khi chốt chất lượng. Giá Standard trả phí: 0,10 USD/triệu token đầu vào, 0,40 USD/triệu token đầu ra [1].
- Zod:** kiểm tra cấu trúc và kiểu dữ liệu mô hình trả về trước khi hiển thị/lưu; không coi cấu trúc hợp lệ là bằng chứng nội dung đúng.
- Git + GitHub:** quản lý mã, tài liệu và phân công qua issue, giúp truy vết thay đổi kế hoạch; demo chạy cục bộ để chưa cần chi phí hosting.

Dự toán: một lượt giả định 2.000 token vào và 500 token ra tốn 0,0004 USD; 5.000 lượt khoảng 2 USD. Dành tối đa 5 USD cho thử nghiệm, kể cả lượt thử lại; theo dõi token thực tế và dừng gọi khi chạm ngân sách ứng dụng. Ước tính chưa gồm thuế, tỷ giá, hosting và không dùng công cụ tìm kiếm trả phí. Dùng dữ liệu mô phỏng/đã ẩn danh khi thử nghiệm.

[1] Google, Gemini Developer API pricing, mục Gemini 2.5 Flash-Lite / Standard, kiểm tra 07/10/2026: <https://ai.google.dev/gemini-api/docs/pricing> — cần kiểm tra lại trước triển khai.